

Exercise 0

Transaction Summary			
=====			
Install 42 Packages			
Total download size: 102 M			
Installed size: 316 M			
Downloading Packages:			
(1/42): annobin-plugin-gcc-12.69-1.amzn2023.0.1.x86_64.rpm	31 MB/s	974 kB	00:00
(2/42): annobin-docs-12.69-1.amzn2023.0.1.noarch.rpm	2.6 MB/s	93 kB	00:00
(3/42): Lmod-8.4.23-1.amzn2023.0.3.x86_64.rpm	5.2 MB/s	218 kB	00:00
(4/42): gc-8.0.4-5.amzn2023.0.2.x86_64.rpm	2.9 MB/s	105 kB	00:00
(5/42): cpp-11.5.0-5.amzn2023.0.5.x86_64.rpm	34 MB/s	11 MB	00:00
(6/42): gcc-c++-11.5.0-5.amzn2023.0.5.x86_64.rpm	33 MB/s	15 MB	00:00
(7/42): gcc-plugin-annobin-11.5.0-5.amzn2023.0.5.x86_64.rpm	1.5 MB/s	38 kB	00:00
(8/42): gcc-gfortran-11.5.0-5.amzn2023.0.5.x86_64.rpm	40 MB/s	15 MB	00:00
(9/42): glibc-devel-2.34-231.amzn2023.0.5.x86_64.rpm	187 kB/s	33 kB	00:00
(10/42): gcc-11.5.0-5.amzn2023.0.5.x86_64.rpm	37 MB/s	36 MB	00:00
(11/42): glibc-headers-x86-2.34-231.amzn2023.0.5.noarch.rpm	1.3 MB/s	439 kB	00:00
(12/42): guile22-2.2.7-2.amzn2023.0.3.x86_64.rpm	17 MB/s	6.4 MB	00:00
(13/42): infinipath-psm-3.3-26_g604758e_open.6.amzn2023.3.0.3.x86_64.rpm	3.2 MB/s	198 kB	00:00
(14/42): hwloc-libs-2.4.1-3.amzn2023.0.6.x86_64.rpm	24 MB/s	2.1 MB	00:00
(15/42): libfabric-1.14.0-2.amzn2023.0.2.x86_64.rpm	23 MB/s	860 kB	00:00
(16/42): kernel6.18-headers-6.18.51-120.162.amzn2023.x86_64.rpm	28 MB/s	1.6 MB	00:00
(17/42): libgfortran-14.2.1-7.amzn2023.0.2.x86_64.rpm	19 MB/s	963 kB	00:00
(18/42): libibumad-48.0-1.amzn2023.0.1.x86_64.rpm	896 kB/s	29 kB	00:00
(19/42): libmpc-1.2.1-2.amzn2023.0.2.x86_64.rpm	2.3 MB/s	62 kB	00:00
(20/42): libpsm2-11.2.86-8.amzn2023.0.2.x86_64.rpm	8.6 MB/s	206 kB	00:00
(21/42): libquadmath-14.2.1-7.amzn2023.0.2.x86_64.rpm	6.8 MB/s	210 kB	00:00
(22/42): libquadmath-devel-11.5.0-5.amzn2023.0.5.x86_64.rpm	5.5 MB/s	176 kB	00:00
(23/42): librdmacm-48.0-1.amzn2023.0.1.x86_64.rpm	2.3 MB/s	73 kB	00:00
(24/42): libtool-ltdl-2.4.7-1.amzn2023.0.3.x86_64.rpm	1.3 MB/s	38 kB	00:00

```
[ec2-user@ip-172-31-20-184 lab8]$ mpirun --version
mpicxx -o HelloMPI HelloMPI.c
mpirun --oversubscribe -np 4 ./HelloMPI
mpirun (Open MPI) 4.1.2
```

```
Report bugs to http://www.open-mpi.org/community/help/
Just a normal process From rank 3 machine ip-172-31-20-184.ec2.internal
Just a normal process From rank 1 machine ip-172-31-20-184.ec2.internal
Just a normal process From rank 2 machine ip-172-31-20-184.ec2.internal
Hello World! From rank 0 machine ip-172-31-20-184.ec2.internal
[ec2-user@ip-172-31-20-184 lab8]$
```

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicxx -o messages1 messages1.cc
mpirun --oversubscribe -np 4 ./messages1
mpicxx -o message2 message2.cc
mpirun --oversubscribe -np 4 ./message2
Just a normal process From rank 0 machine ip-172-31-20-184.ec2.internal
Just a normal process From rank 2 machine ip-172-31-20-184.ec2.internal
SEnding message to computer 3 from computer 1
in computer 3 the value of y is 30
Just a normal process From rank 2 machine ip-172-31-20-184.ec2.internal
Just a normal process From rank 0 machine ip-172-31-20-184.ec2.internal
SEnding message to computer 3 from computer 1
in computer 3 the value of y is printed
0 10 20 30 40 50 60 70 80 90 [ec2-user@ip-172-31-20-184 lab8]$ |
```

Open MPI 4.1.2 was installed on a new EC2 instance (Amazon Linux 2023, 2 vCPUs) together with gcc and gcc-c++. The three in-class programs from the SE4060-MPI-Part1 repository were compiled with mpicxx and run with 4 processes. HelloMPI prints one line from every rank. In messages1.cc rank 1 sends $x = 30$ to rank 3 with MPI_Ssend and rank 3 receives it into y with MPI_Recv; ranks 0 and 2 only print their rank. message2.cc is the same, but an array of 10 integers (0, 10, ... 90) is sent in a single message.

The order of the lines changes from run to run because the processes run at the same time.

Exercise 1

The repository PC-LAB8-IT24103322 was created on my GitHub account (it24103322). The answer report for each exercise is pushed into its own folder, Exercise00 to Exercise07.

Exercise 2

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicc -o sum_mpi sum_mpi.c
mpirun --oversubscribe -np 4 ./sum_mpi
Rank 3: summed 7500001 to 10000000 => local_sum = 21875001250000
Rank 1: summed 2500001 to 5000000 => local_sum = 9375001250000
Rank 2: summed 5000001 to 7500000 => local_sum = 15625001250000
Rank 0: summed 1 to 2500000 => local_sum = 3125001250000

[Sum] Processes = 4
[Sum] Total sum = 50000005000000
[Sum] Expected = 50000005000000
[Sum] Correct? = YES
[Sum] Time = 0.014428 sec
[ec2-user@ip-172-31-20-184 lab8]$
```

The range 1 to 10,000,000 is divided equally between the processes. Each process works out its own first and last number from its rank ($\text{chunk_size} = N / \text{size}$, the last rank also takes the remainder) and adds them into a long long `local_sum`. No array is needed, so nothing has to be sent to the workers. Every process except root sends its `local_sum` to rank 0 with `MPI_Send`, and root receives them one by one with `MPI_Recv` (`source = r`) and adds them to its own sum.

Total sum = 50000005000000, which matches the expected value $N(N + 1) / 2$ (Correct? = YES). Time = 0.014428 sec with 4 processes.

Exercise 3

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicc -o pi_mpi pi_mpi.c -lm
mpirun --oversubscribe -np 4 ./pi_mpi
Rank 0: trials = 2500000, count = 1962662
Root received count = 1962547 from rank 1
Root received count = 1963912 from rank 2
Root received count = 1964200 from rank 3

[Pi] Processes = 4
[Pi] Trials = 10000000
[Pi] Points inside = 7853321
[Pi] Estimate of pi = 3.1413284000
[Pi] Error = 0.0002642536
[Pi] Time = 0.282547 sec
[ec2-user@ip-172-31-20-184 lab8]$ |
```

The 10,000,000 trials are divided between the processes (N / size each, the remainder goes to the first ranks). Each process seeds `rand()` with `SEED + rank` so that every process generates different points, counts how many of its points fall inside the quarter circle ($x^2 + y^2 \leq 1$) and sends only that count to root with `MPI_Send`. Root receives the counts in rank order with `MPI_Recv` (`source = r`) and calculates $\text{pi} = 4 \times \text{total_count} / N$.

With 4 processes 7,853,321 of the 10,000,000 points were inside, which gives $\text{pi} = 3.1413284$ (error = 0.00026). Time = 0.282547 sec. The estimate changes slightly with the number of processes because a different set of seeds is used.

Exercise 4

```
[ec2-user@ip-172-31-20-184 lab8]$ nproc
chmod +x run_timings.sh
./run_timings.sh
python3 timing_table.py
2
Run 1 of 10 finished
Run 2 of 10 finished
Run 3 of 10 finished
Run 4 of 10 finished
Run 5 of 10 finished
Run 6 of 10 finished
Run 7 of 10 finished
Run 8 of 10 finished
Run 9 of 10 finished
Run 10 of 10 finished
Saved 320 timings to timings.csv
Median time in seconds (10 runs each)
Processes    pi_anysource    pi_bsend    pi_mpi    sum_mpi
1            0.352585      0.351998    0.351410    0.025367
2            0.264409      0.264343    0.263775    0.013931
3            0.269188      0.276639    0.270823    0.012126
4            0.271872      0.300792    0.273346    0.014655
5            0.276270      0.276176    0.269838    0.016733
6            0.274130      0.278072    0.273563    0.017529
7            0.270494      0.272082    0.270859    0.016399
8            0.261719      0.253722    0.252594    0.009369

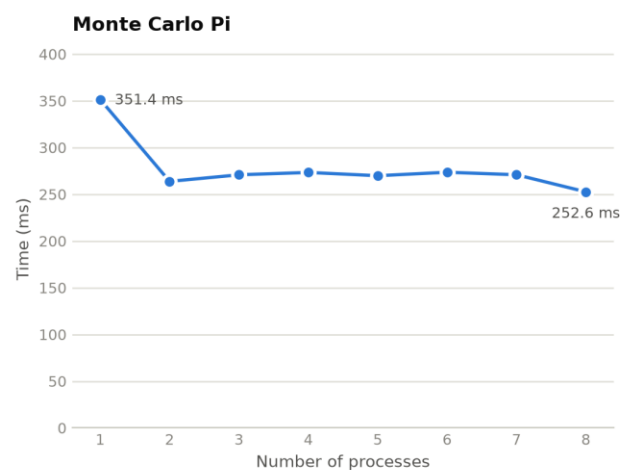
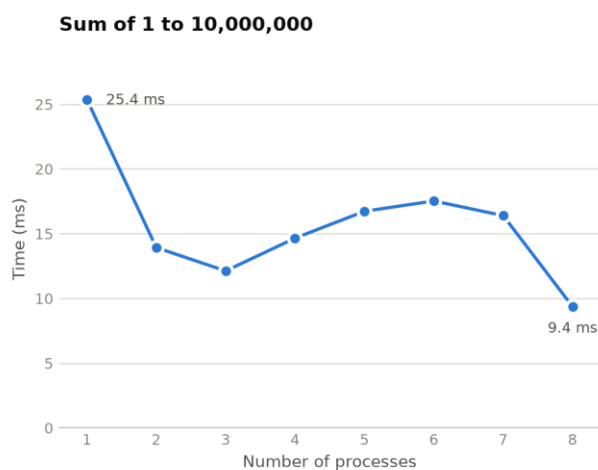
Speedup (T1 / Tp)
Processes    pi_anysource    pi_bsend    pi_mpi    sum_mpi
1            1.00x          1.00x      1.00x      1.00x
2            1.33x          1.33x      1.33x      1.82x
3            1.31x          1.27x      1.30x      2.09x
4            1.30x          1.17x      1.29x      1.73x
5            1.28x          1.27x      1.30x      1.52x
6            1.29x          1.27x      1.28x      1.45x
7            1.30x          1.29x      1.30x      1.55x
8            1.35x          1.39x      1.39x      2.71x

[ec2-user@ip-172-31-20-184 lab8]$
```

The lab sheet says Exercise 1 and Exercise 2; the two programs are the sum (Exercise 2) and the Pi calculation (Exercise 3). Both were run with 1 to 8 processes using run_timings.sh. Every run was repeated 10 times and the median time is used, so that one slow run does not change the result. Speedup = T_1 / T_p . The EC2 instance has only 2 vCPUs (nproc = 2), so mpirun needs the oversubscribe option and with more than 2 processes the processes have to share the CPUs.

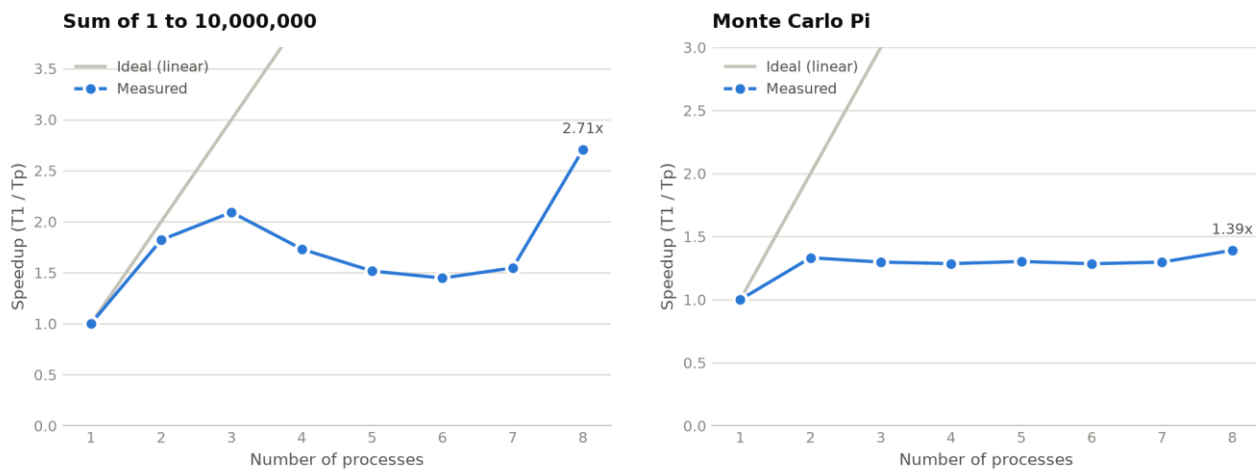
Processes	Sum time (ms)	Sum speedup	Pi time (ms)	Pi speedup
1	25.37	1.00×	351.41	1.00×
2	13.93	1.82×	263.77	1.33×
3	12.13	2.09×	270.82	1.30×
4	14.65	1.73×	273.35	1.29×
5	16.73	1.52×	269.84	1.30×
6	17.53	1.45×	273.56	1.28×
7	16.40	1.55×	270.86	1.30×
8	9.37	2.71×	252.59	1.39×

1. Plot a graph that shows Time vs Number of Processors



The time drops when the second process is added (sum: 25.37 ms → 13.93 ms, Pi: 351.41 ms → 263.77 ms). After that the Pi time stays almost the same from 2 to 7 processes (264 to 274 ms) and is only slightly lower with 8 (252.59 ms). There are only 2 vCPUs, so with more than 2 processes the processes take turns on the CPUs and the total amount of work is the same. The sum time goes up and down between 9.4 ms and 17.5 ms because the whole run is very short.

2. Plot a graph that shows Speedup



The Pi program reaches 1.33x with 2 processes and stays at about 1.3x up to 8 processes (1.39x), far below the ideal line. More processes than vCPUs cannot give more speedup. It is below 2x even with 2 processes, most likely because the 2 vCPUs are two hardware threads of one physical core and not two full cores.

The sum reaches 1.82x with 2 processes. Its values above 2 processes (2.09x at 3, 1.45x at 6, 2.71x at 8) are not a real speedup on 2 vCPUs. The run takes only 10 to 25 ms and the time is measured on root from the moment root leaves MPI_Barrier, so when the processes are taking turns on the CPUs the other processes can already have done part of their loop before the timer on root starts.

Exercise 5

1. Write a simple program where the source and the destination doesn't match. You can take either message1.cc or message2.cc and modify the program. What happens in this situation?

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicxx -o mismatch mismatch.cc
mpirun --timeout 10 --oversubscribe -np 3 ./mismatch; echo "exit status = $?"
mpirun --oversubscribe -np 2 ./mismatch
Process 1 waiting for a message from process 2
Process 0 sent 42 to process 1
-----
The user-provided time limit for job execution has been reached:

Timeout: 10 seconds

The job will now be aborted. Please check your code and/or
adjust/remove the job execution time limit (as specified by --timeout
command line option or MPIEXEC_TIMEOUT environment variable).
-----
exit status = 110
Process 0 sent 42 to process 1
Process 1 waiting for a message from process 2
[ip-172-31-20-184:36903] *** An error occurred in MPI_Recv
[ip-172-31-20-184:36903] *** reported by process [3239575553,1]
[ip-172-31-20-184:36903] *** on communicator MPI_COMM_WORLD
[ip-172-31-20-184:36903] *** MPI_ERR_RANK: invalid rank
[ip-172-31-20-184:36903] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[ip-172-31-20-184:36903] *** and potentially your MPI job)
[ec2-user@ip-172-31-20-184 lab8]$
```

message1.cc was changed (mismatch.cc) so that rank 0 still sends the number to rank 1, but rank 1 calls MPI_Recv with source = 2 instead of 0. With 3 processes MPI_Send on rank 0 returns (the message is small, so it is buffered) and "Process 0 sent 42 to process 1" is printed, but rank 1 is waiting for a message from rank 2 and rank 2 never sends one. MPI_Recv is a blocking call and the message from rank 0 does not match the requested source, so rank 1 blocks forever. There is no error message; the program just hangs and never finishes. It was stopped after 10 seconds with the timeout option of mpirun (exit status = 110).

When the same program is run with only 2 processes, rank 2 does not exist, so MPI_Recv fails immediately with MPI_ERR_RANK: invalid rank and the job is aborted. With MPI_Ssend instead of MPI_Send the sender blocks as well, because a synchronous send only returns after the matching receive has started.

2. Rewrite message2.cc using Buffered Send (BSend) ensure that you do not replace the variables that you are using to send the data.

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicxx -o message2 message2.cc
mpirun --oversubscribe -np 2 ./message2
mpicxx -o message2_bsend message2_bsend.cc
mpirun --oversubscribe -np 2 ./message2_bsend
Process 0 sent 0
Process 0 sent 10
Process 0 sent 20
Process 1 received 0
Process 1 received 10
Process 1 received 20
Process 0 sent 0
Process 0 sent 10
Process 0 sent 20
Process 1 received 0
Process 1 received 10
Process 1 received 20
[ec2-user@ip-172-31-20-184 lab8]$ |
```

MPI_Send was replaced with MPI_Bsend (message2_bsend.cc). Before the loop rank 0 attaches its own buffer with MPI_Buffer_attach. The buffer is sized for all three messages, $3 \times (\text{MPI_Pack_size of one MPI_INT} + \text{MPI_BSEND_OVERHEAD})$, so a later message never overwrites one that is still waiting in the buffer. The first six lines of the output are from the original message2.cc and the last six are from the buffered version.

The same variable number is still used for all three sends. MPI_Bsend copies the value into the attached buffer before it returns, so number can be changed in the next iteration without affecting the messages that were already sent. To show this, rank 1 sleeps for 1 second before it starts receiving: all three sends finish first and rank 1 still receives 0, 10 and 20. MPI_Buffer_detach is called after the loop, which waits until all the buffered messages have been delivered before the buffer is freed.

Exercise 6

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicc -o pi_anysource pi_anysource.c -lm
mpirun --oversubscribe -np 4 ./pi_anysource
Rank 0: trials = 2500000, count = 1962662
Root received count = 1964200 from rank 3
Root received count = 1962547 from rank 1
Root received count = 1963912 from rank 2

[Pi ANY_SOURCE] Processes = 4
[Pi ANY_SOURCE] Trials = 10000000
[Pi ANY_SOURCE] Points inside = 7853321
[Pi ANY_SOURCE] Estimate of pi = 3.1413284000
[Pi ANY_SOURCE] Error = 0.0002642536
[Pi ANY_SOURCE] Time = 0.265539 sec
[ec2-user@ip-172-31-20-184 lab8]$ |
```

In `pi_anysource.c` the source argument of `MPI_Recv` on root was changed from `r` to `MPI_ANY_SOURCE` (the tag is still 0), and an `MPI_Status` is used to read which rank the message came from (`status.MPI_SOURCE`). Root now takes the counts in the order they arrive instead of rank order: in this run rank 3 arrived first, then rank 1 and rank 2, while the original program always receives 1, 2, 3.

The result is identical to Exercise 3 (7,853,321 points inside, $\pi = 3.1413284$) because the same seeds are used and the order of the additions does not change the total.

Processes	Exercise 3 (source = r)	MPI_ANY_SOURCE
1	351.41 ms	352.58 ms
2	263.77 ms	264.41 ms
3	270.82 ms	269.19 ms
4	273.35 ms	271.87 ms
5	269.84 ms	276.27 ms
6	273.56 ms	274.13 ms
7	270.86 ms	270.49 ms
8	252.59 ms	261.72 ms

The times (median of 10 runs) are almost the same as Exercise 3 for every process count; the largest difference is 3.6 % at 8 processes (261.72 ms against 252.59 ms). Root still has to wait for every count and each process sends only one 8-byte message. `MPI_ANY_SOURCE` helps when the processes finish at different times and root can use each result as soon as it arrives; here every process has the same amount of work.

Exercise 7

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicc -o pi_bsend pi_bsend.c -lm
mpirun --oversubscribe -np 4 ./pi_bsend
Rank 0: trials = 2500000, count = 1962662
Root received count = 1962547 from rank 1
Root received count = 1963912 from rank 2
Root received count = 1964200 from rank 3

[Pi Bsend] Processes = 4
[Pi Bsend] Trials = 10000000
[Pi Bsend] Points inside = 7853321
[Pi Bsend] Estimate of pi = 3.1413284000
[Pi Bsend] Error = 0.0002642536
[Pi Bsend] Time = 0.272962 sec
[ec2-user@ip-172-31-20-184 lab8]$ |
```

In `pi_bsend.c` the workers send their count with `MPI_Bsend` instead of `MPI_Send`. Each worker attaches a buffer of `MPI_Pack_size(1, MPI_LONG_LONG) + MPI_BSEND_OVERHEAD` bytes with `MPI_Buffer_attach`, calls `MPI_Bsend` and then `MPI_Buffer_detach`, which waits until the message has left the buffer. Root is unchanged and still receives with `MPI_ANY_SOURCE` (in this run the counts happened to arrive in rank order). The result is the same again ($\pi = 3.1413284$).

Processes	Exercise 3 (MPI_Send)	Exercise 6 (ANY_SOURCE)	Exercise 7 (MPI_Bsend)
1	351.41 ms	352.58 ms	352.00 ms
2	263.77 ms	264.41 ms	264.34 ms
3	270.82 ms	269.19 ms	276.64 ms
4	273.35 ms	271.87 ms	300.79 ms
5	269.84 ms	276.27 ms	276.18 ms
6	273.56 ms	274.13 ms	278.07 ms
7	270.86 ms	270.49 ms	272.08 ms
8	252.59 ms	261.72 ms	253.72 ms

Buffered send made no real difference to the time either. The only larger difference from Exercise 3 is at 4 processes (300.79 ms against 273.35 ms, 10 %), and it does not appear at the other process counts.

`MPI_Bsend` returns as soon as the message is copied into the buffer, so a worker never waits for root, but the message is only one long long and `MPI_Send` already sends such a small message without waiting.

`MPI_Bsend` is useful when the message is large, or when the sender must continue before the receiver is ready.